

Analysing the Impact of DoS Attacks on System Availability

Tool: Wireshark

Theory

A Denial-of-Service (DoS) attack overwhelms a target system with a flood of traffic or requests, making it unavailable to legitimate users. A common example is a SYN flood where the attacker sends a large number of TCP SYN packets without completing the handshake, exhausting the server's connection table. Wireshark is used to capture and analyse the flood traffic to observe its impact. This experiment is performed only within an isolated lab network on authorised virtual machines.

Steps

1. Start Kali Linux and Metasploitable 2 on a Host-Only network in VirtualBox. Install hping3 on Kali if not already present. Run: `sudo apt install hping3 -y`
 2. Open Wireshark on Kali. Run: `sudo wireshark` — Select the eth0 interface and click Start Capture to begin recording traffic.
 3. Open a second terminal. Verify the target is reachable before the attack. Run: `ping -c 3 192.168.56.102` — Note the normal response times.
 4. Start a SYN flood against the target web service on port 80. Run: `sudo hping3 -S -p 80 --flood 192.168.56.102` — This sends TCP SYN packets as fast as possible to port 80 on the target.
 5. Observe Wireshark during the flood. Apply the filter: `tcp.flags.syn == 1` — The packet list fills rapidly with SYN packets all originating from Kali. Notice the packet rate in the bottom status bar increasing dramatically.
 6. Go to Statistics menu in Wireshark and select IO Graph. The graph shows a sharp spike in traffic volume during the flood, visually demonstrating the attack impact.
 7. Press Ctrl+C in the hping3 terminal to stop the flood. Try pinging the target again. Run: `ping -c 3 192.168.56.102` — Compare response times before and after the attack.
 8. In Wireshark click File > Save As and save the capture as `dos_capture.pcapng` on the Desktop for documentation.
-

Conclusion

Wireshark captured and visualised the SYN flood traffic, clearly showing the sudden spike in packet volume during the DoS attack. The experiment demonstrated how flooding a service with incomplete connection requests degrades availability and highlighted the need for rate limiting and SYN cookie protection on servers.